

Tech Stack Reference

Layer	Technology	Version
Framework	Flutter / Dart	SDK ^3.10.8
On-Device AI — Stage 1	TensorFlow Lite (YOLOv8n)	tflite_flutter ^0.12.1
On-Device AI — Stage 2	TensorFlow Lite (MobileNetV3-Small)	tflite_flutter ^0.12.1
Auth	Firebase Anonymous Auth	firebase_auth ^5.5.4
Cloud DB	Cloud Firestore	cloud_firestore ^5.6.7
Local DB	Hive (NoSQL key-value)	hive ^2.2.3
Location	Geolocator	^13.0.4
Connectivity	connectivity_plus + http	^6.1.4 / ^1.4.0
Maps	flutter_map (OpenStreetMap)	^8.3.0
Image	image (decode/resize/crop)	^4.2.0
File I/O	file_picker / image_picker	^11.0.2 / ^1.1.2

Part 1 — Step-by-Step Runtime Documentation

Phase 0 — Cold Start & Service Initialization (main.dart)

The process entry point is `main()` in `lib/main.dart`. This is a **synchronous-blocking** initialization sequence — the app does not render any UI until all `await` calls resolve.

Step 0.1 — Flutter Engine Bootstrap

dart

```
WidgetsFlutterBinding.ensureInitialized();
```

What happens: Binds the Flutter widget engine to the native platform channel before any async platform calls can be made. Without this, any `await` on platform plugins before `runApp()` would throw. No UI is rendered yet.

State change: Platform channels become available.

Step 0.2 — Firebase Platform Initialization

dart

```
await Firebase.initializeApp(  
  options: DefaultFirebaseOptions.currentPlatform,  
);
```

What happens: Reads `firebase_options.dart` (generated by FlutterFire CLI) and initializes the Firebase SDK with platform-specific API keys, project IDs, and app IDs. Establishes the Firebase app instance used by all subsequent Firebase services.

Error handling: If Firebase fails to initialize (e.g., corrupted `google-services.json`), an unhandled exception crashes the app at startup. No graceful recovery — this is treated as a fatal configuration error.

Step 0.3 — Anonymous Authentication

dart

```
try {
  await FirebaseAuth.instance.signInAnonymously();
} catch (e) {
  debugPrint("Failed to sign in anonymously: $e");
}
```

What happens: Creates (or retrieves a cached) anonymous Firebase user. This UID is required by Firestore security rules to permit `sightings` collection writes. The token is cached by the Firebase SDK and auto-refreshed.

Error handling: Wrapped in `try/catch`. If sign-in fails (no internet at cold start, Firebase outage), the error is silently logged via `debugPrint`. The app continues — offline-first write queue handles the case where auth is unavailable during upload.

State change: `FirebaseAuth.instance.currentUser` becomes non-null.

Step 0.4 — TFLite Model Loading (`TfliteService.init()`)

dart

```
await TfliteService().init();
```

Singleton check: `if (!_initialized) return;` — idempotent, safe to call multiple times.

Internal execution:

- `rootBundle.load('Model/yolov8n_detector.tflite')`
→ Loads raw bytes from Flutter asset bundle into memory
→ `Interpreter.fromBuffer(bytes)` → `_detectorInterpreter`
- `rootBundle.load('Model/mobilenetv3_mosquito_classifier.tflite')`
→ `Interpreter.fromBuffer(bytes)` → `_classifierInterpreter`
- `rootBundle.loadString('Model/labels_detector.txt')`
→ split on `[\r\n]+` → filter empty → `_detectorLabels: List<String>`
- `rootBundle.loadString('Model/labels_classifier.txt')`
→ split on `[\r\n]+` → filter empty → `_classifierLabels: List<String>`
→ `["Aegypti", "Albopictus", "Anopheles", "Culex", "Non-Mosquito"]`
- `_detectorInterpreter.getOutputTensors().first.shape`
→ if `shape[1] > shape[2]`: `_outputAnchorMajor = true`
→ `_numAnchors` = anchor dimension count (typically 8400 for YOLOv8n)
→ `_numChannels` = channel dimension count
→ `_initialized = true`

Error handling: If `rootBundle.load` fails (asset not bundled), a `FlutterError` is thrown — fatal at startup. Models are required assets; no fallback exists.

Performance note: Both interpreter allocations occur here, not at inference time. This makes the first `predict()` call fast.

Step 0.5 — Hive Local Database Init

dart

```
await Hive.initFlutter();
```

What happens: Resolves the platform-appropriate documents directory via `path_provider` and configures Hive to use it as the root storage path.

Step 0.6 — History Service Init (`HistoryService.init()`)

dart

```
await HistoryService().init();
```

Internal execution:

```
1. _historyBox = await Hive.openBox('historyBox')
   → Opens or creates persistent Hive box

2. _loadFromBox()
   → _records.clear()
   → for each key in _historyBox.keys:
       json = _historyBox.get(key) as Map
       _records.add(ScanRecord.fromJson(json))
   → _records.sort((a, b) => b.date.compareTo(a.date))
      [newest-first ordering for history UI]
```

State change: `HistoryService()._records` populated in-memory. Max capacity: 12 records (`_maxRecords`).

Step 0.7 — Tutorial Service Init (`TutorialService.init()`)

dart

```
await TutorialService().init();
```

Internal execution:

```
_box = await Hive.openBox('tutorial_prefs')
```

State change: `TutorialService()._box` available. `hasSeenTutorial` getter now readable: `_box.get('has_seen_tutorial', defaultValue: false)`.

Step 0.8 — Connectivity Monitoring Init (`ConnectivityService.init()`)

dart

```
ConnectivityService().init();
```

Note: **not** awaited — non-blocking.

Internal execution:

```
Connectivity().onConnectivityChanged.listen(
  (List<ConnectivityResult> results) async {
    bool hasInternet = await hasInternetAccess();
    _connectivityStreamController.add(hasInternet);
  }
);
```

State change: Background stream listener active. Any network interface change triggers `hasInternetAccess()` and broadcasts the result to all subscribers.

Step 0.9 — Upload Queue Service Init (`UploadQueueService.init()`)

dart

```
await UploadQueueService().init();
```

Internal execution:

1. `_queueBox = await Hive.openBox('upload_queue')`
2. `_deadLetterBox = await Hive.openBox('dead_letter_queue')`
3. `_drainTimer = Timer.periodic(Duration(seconds: 30), (_) => _drainQueue())`
→ Background timer: attempts queue drain every 30 seconds
4. `_connectivitySubscription = ConnectivityService().connectivityStream`
`.listen((hasInternet) {`
 `if (hasInternet) _drainQueue(); // immediate drain on reconnect`
`});`
5. `_drainQueue()` // attempt upload of any queued items from prior sessions

State change: Two Hive boxes open, periodic drain timer active, connectivity subscription active.

Step 0.10 — App Render

dart

```
runApp(const DengueLensApp());
```

`DengueLensApp.build()` constructs:

dart

```
MaterialApp(  
  title: 'Dengue Lens - Dengue Vector Detection',  
  debugShowCheckedModeBanner: false,  
  theme: ThemeData(  
    useMaterial3: true,  
    fontFamily: 'Roboto',  
    colorScheme: ColorScheme.fromSeed(  
      seedColor: Color(0xFF2ECC71),  
      brightness: Brightness.light,  
    ),  
  ),  
  home: DengueLensHome(),  
)
```

Phase 1 — Home Screen Mount (`dengue_lens_home.dart`)

Step 1.1 — Widget Tree Construction

`DengueLensHomeState` is created. Initial state:

```
dart

bool _isProcessing = false;
String? _processingImagePath = null;
bool _showTutorial = false;

// GlobalKey registered for tutorial spotlight targeting
final GlobalKey _scanButtonKey = GlobalKey();
final GlobalKey _uploadButtonKey = GlobalKey();
final GlobalKey _navBarKey = GlobalKey();
final GlobalKey _fabKey = GlobalKey();
```

Step 1.2 — `initState()` — Tutorial Gate Check

```
dart

@override
void initState() {
  super.initState();
  WidgetsBinding.instance.addPostFrameCallback((_) {
    if (!TutorialService().hasSeenTutorial && mounted) {
      setState(() => _showTutorial = true);
    }
  });
}
```

Why `addPostFrameCallback`? The callback fires after the first frame renders, ensuring all `GlobalKey`-attached widgets have their `RenderObject`'s positioned before `TutorialOverlay` tries to find their screen coordinates via `findRenderObject()`.

State change: If `has_seen_tutorial == false` in Hive → `_showTutorial = true` → `setState()` triggers rebuild → `TutorialOverlay` widget is conditionally rendered in the `Stack`.

UI rendered:

- `HomeHeader` — brand logo + title + notification bell icon
- `HeroSection` — "Identify Mosquito & Assess Risk" text
- `ScanButton` (`key: _scanButtonKey`) — circular green camera button
- `UploadButton` (`key: _uploadButtonKey`) — outlined gallery button
- `HealthTipCard` — bite treatment tips with "Read More" modal
- `DailyTipCard` — DEET tip banner
- `FloatingActionButton` (`key: _fabKey`) — replay tutorial
- `NavigationBar` (`key: _navBarKey`) — 5 bottom destinations

Phase 2 — Tutorial Onboarding (tutorial_overlay.dart)

Triggered only when `_showTutorial == true`.

Step 2.1 — TutorialOverlay.initState()

```
dart

_steps = [
  _TutorialStep(title: 'Welcome to Dengue Lens!', targetKey: null), // Step 0: full-screen
  _TutorialStep(title: 'Scan a Mosquito', targetKey: scanKey), // Step 1
  _TutorialStep(title: 'Upload from Gallery', targetKey: uploadKey), // Step 2
  _TutorialStep(title: 'Explore the App', targetKey: navBarKey), // Step 3
  _TutorialStep(title: 'Replay Anytime', targetKey: fabKey), // Step 4
];

_animController = AnimationController(
  vsync: this,
  duration: Duration(milliseconds: 350),
);

_fadeAnimation = CurvedAnimation(parent: _animController, curve: Curves.easeInOut);
_animController.forward(); // fade-in first step
```

Step 2.2 — Spotlight Rendering Per Step

For each step with a non-null `targetKey`:

```
dart

Rect? _getTargetRect(GlobalKey key) {
  final renderObject = key.currentContext?.findRenderObject();
  if (renderObject is RenderBox && renderObject.hasSize) {
    final offset = renderObject.localToGlobal(Offset.zero);
    return offset & renderObject.size; // Rect from offset + size
  }
  return null; // key not yet mounted + no spotlight
}
```

`_SpotlightPainter.paint()` — draws a full-screen semi-transparent black overlay (`Color(0xFF000000).withOpacity(0.72)`), then subtracts a cutout `Path` (oval for circular targets like the ScanButton, rounded-rect for others) using `Path.combine(PathOperation.difference, fullScreen, cutout)`.

`_PulsingRing.initState()` — starts a 1500ms repeating `AnimationController`:

```
dart

_scaleAnim: Tween(begin: 1.0, end: 1.15) → easeOut
_opacityAnim: Tween(begin: 0.6, end: 0.0) → easeOut
_controller..repeat()
```

Step 2.3 — Step Navigation

dart

```
void _goToStep(int step) {
  _animController.reverse().then((_) { // fade out current step (350ms)
    if (!mounted) return;
    setState(() => _currentStep = step); // update step index
    _animController.forward(); // fade in new step (350ms)
  });
}

void _next() {
  if (_currentStep < _steps.length - 1) {
    _goToStep(_currentStep + 1);
  } else {
    _finish();
  }
}

void _skip() => _finish();
```

Step 2.4 — Tutorial Completion (_finish())

dart

```
void _finish() {
  TutorialService().markTutorialSeen(); // Hive.put('has_seen_tutorial', true)
  _animController.reverse().then((_) {
    if (mounted) widget.onDismiss(); // parent: setState(_showTutorial = false)
  });
}
```

State change: Tutorial flag persisted to Hive. `TutorialOverlay` widget removed from the `Stack`.

Edge case — Tutorial replay: User taps the `FloatingActionButton` → `_showReplayConfirmation()` → `AlertDialog` shown → User confirms → `TutorialService().resetTutorial()` (`Hive.put('has_seen_tutorial', false)`) → `setState(() => _showTutorial = true)` → overlay re-renders.

Phase 3 — Feature A: Camera Scan (`_captureImageFromCamera`)

Trigger: User taps the green `ScanButton`.

Step 3.1 — Re-entrancy Guard

```
dart

if (_isProcessing) return;
```

Prevents double-tap or concurrent analysis. If `_isProcessing == true`, the method exits immediately.

Step 3.2 — Native Camera Intent

```
dart

final ImagePicker picker = ImagePicker();
final XFile? photo = await picker.pickImage(source: ImageSource.camera);
```

Platform behavior:

- **Android:** Launches `ACTION_IMAGE_CAPTURE` intent to the system camera app. Returns `XFile` with a temp file path.
- **iOS:** Presents `UIImagePickerController` with camera source.
- **Windows/Desktop:** `image_picker` does not support camera on desktop. This button may be functionally inactive or throw.

Edge cases:

- `photo == null` — user pressed Back/Cancel → method returns, no state change, no UI update.
- Camera permission denied (first time) — OS shows permission dialog. If denied, `pickImage` throws or returns null depending on platform. No explicit in-app permission pre-check is coded; the OS handles it.

Step 3.3 — Loading Overlay Activation

```
dart

setState(() {
  _isProcessing = true;
  _processingImagePath = photo.path;
});
```

UI change: The `Stack` conditionally renders a full-screen black overlay (`Colors.black.withOpacity(0.65)`) containing:

- `Image.file(File(_processingImagePath!))` — 180×180 thumbnail preview
- `CircularProgressIndicator(color: Color(0xFF2ECC71), strokeWidth: 4)`
- Text: "Analysing image..." / "Running two-stage detection"

This overlay blocks all touch input because it sits on top of the entire widget tree.

Step 3.4 — TFLite Inference (Two-Stage Pipeline)

```
dart

final prediction = await TfliteService().predict(File(photo.path));
```

→ See **Phase 5** for granular inference breakdown.

Step 3.5 — Navigation to Results

dart

```
final isPositive = prediction.isDengueVector;
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => ResultScreen(
      imagePath: File(photo.path),
      testDate: DateTime.now(),
      result: isPositive ? "positive" : "negative",
      confidence: prediction.confidence,
      sampleType: "Mosquito Image",
      mosquitoType: prediction.displayName,
      boundingBox: _primaryBoundingBox(prediction),
      detections: prediction.detections,
      imageSize: prediction.imageSize,
      savedDetectionCount: prediction.detections.length,
    ),
  ),
);
```

_primaryBoundingBox(PredictionResult prediction):

dart

```
// Prioritize dengue vectors over non-vectors; highest confidence wins.
final vectors = detections.where((d) => d.isDengueVector).toList();
if (vectors.isNotEmpty) {
  vectors.sort((a, b) => b.confidence.compareTo(a.confidence));
  return vectors.first.boundingBox; // primary box = highest-confidence dengue vector
}
// Fallback: if no dengue vectors, return the highest-confidence detection regardless
final sorted = List<Detection>.from(detections)
  ..sort((a, b) => b.confidence.compareTo(a.confidence));
return sorted.first.boundingBox;
```

Step 3.6 — Cleanup

dart

```
finally {
  if (mounted) setState(() => _isProcessing = false);
}
```

Error path: If `TfliteService().predict()` throws:

dart

```
catch (e) {
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Prediction failed: $e'))
    );
  }
}
```

Loading overlay dismissed, snackbar shown. User can retry.

Phase 4 — Feature B: Gallery Upload (`_pickImageFromGallery`)

Trigger: User taps the `UploadButton`.

Step 4.1 — Re-entrancy Guard

```
dart

if (_isProcessing) return;
```

Step 4.2 — File Picker Dialog

```
dart

FilePickerResult? result = await FilePicker.pickFiles(
  type: FileType.custom,
  allowedExtensions: ['jpg', 'jpeg', 'png', 'gif', 'webp'],
  dialogTitle: 'Select an image',
);
```

Why `FilePicker` instead of `ImagePicker` for gallery? `file_picker ^11.0.2` provides better cross-platform support, especially for Windows desktop, with a proper native file dialog and extension filtering.

Edge cases:

- `result == null` — user cancelled dialog → return, no state change.
- `result.files.single.path == null` — file path unavailable (e.g., cloud file not cached locally) → method falls through with `null` check.

Step 4.3 — Extension Validation

```
dart

final fileExtension = filePath.split('.').last.toLowerCase();
final supportedFormats = ['jpg', 'jpeg', 'png', 'gif', 'webp'];

if (supportedFormats.contains(fileExtension)) {
  // proceed
} else {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Unsupported format. Please select JPEG, PNG, GIF or WebP'),
    ),
  );
}
```

Why client-side validation? `image` package supports these formats. Files outside this set would cause `img.decodeImage()` to return `null` inside `predict()`, throwing a `Exception('Could not decode image')`.

Steps 4.4 → 4.6 — identical to Camera Scan Steps 3.3 → 3.6.

Phase 5 — Two-Stage On-Device AI Inference (`TfLiteService.predict()`)

This is the computational core of Dengue Lens. All inference runs entirely on-device — no network call is made.

Step 5.1 — Image I/O and EXIF Correction

dart

```
final bytes = await imageFile.readAsBytes();
img.Image? original = img.decodeImage(bytes);
if (original == null) {
  throw Exception('Could not decode image: ${imageFile.path}');
}
original = img.bakeOrientation(original);
```

`img.bakeOrientation()` reads the EXIF orientation tag and rotates/flips pixel data so `width`/`height` reflect the visual presentation. Without this, portrait-shot images would be processed sideways, causing detection failures.

Step 5.2 — Stage 1 Input: Letterbox Preprocessing (YOLOv8 standard)

dart

```
static const int _inputSize = 640;

// 1. Compute scale ratio to fit longest edge to 640px
final double ratio = targetSize / math.max(original.width, original.height);
final int newUnpadW = (original.width * ratio).round();
final int newUnpadH = (original.height * ratio).round();

// 2. Resize preserving aspect ratio
final resized = img.copyResize(
  original,
  width: newUnpadW,
  height: newUnpadH,
  interpolation: img.Interpolation.linear,
);

// 3. Create 640×640 gray canvas (YOLO standard padding: 114, 114, 114)
final padded = img.Image(width: targetSize, height: targetSize);
img.fill(padded, color: img.ColorRgb8(114, 114, 114));

// 4. Compute centering offsets
final int dx = ((targetSize - newUnpadW) / 2).round();
final int dy = ((targetSize - newUnpadH) / 2).round();

// 5. Paste resized image centered on gray canvas
img.compositeImage(padded, resized, dstX: dx, dstY: dy);
```

Why letterbox? YOLOv8 requires exactly 640×640 input. Letterboxing preserves aspect ratio (no squishing distortion) and pads with neutral gray — the same gray value used during model training.

Step 5.3 — Tensor Construction (`imageToInputTensor`)

```
dart

// lib/utils/image_utils.dart
List<List<List<List<double>>>> imageToInputTensor(img.Image image) {
  return List.generate(1, (_) =>           // batch dim = 1
    List.generate(image.height, (y) =>      // height = 640
      List.generate(image.width, (x) {      // width = 640
        final pixel = image.getPixel(x, y);
        return [
          pixel.r / 255.0,                  // normalize to [0, 1]
          pixel.g / 255.0,
          pixel.b / 255.0,
        ];
      })),
  ),
);
}

// Output shape: [1][640][640][3] dtype: float32
```

Step 5.4 — YOLOv8 Detector Inference

```
dart

_detectorInterpreter.run(input, output);
// output shape: [1][8400][channels] or [1][channels][8400]
// depending on export format (transposed vs standard)
```

Step 5.5 — Output Shape Adaptation

```
dart

// At init() time, shape was detected:
// _outputAnchorMajor = (outShape[1] > outShape[2])
// _numAnchors = anchor count (typically 8400 for 640px input)
// _numChannels = 4 coords + num_classes (= 5 for single-class "Mosquito")

double _readOutput(List<dynamic> output, int channel, int anchor) {
  return _outputAnchorMajor
    ? (output[0][anchor][channel] as num).toDouble() // [batch][anchor][channel]
    : (output[0][channel][anchor] as num).toDouble(); // [batch][channel][anchor]
}
```

Step 5.6 — YOLO Version Detection & Confidence Parsing

```
dart

// YOLOv5 has explicit objectness score at channel 4 (numChannels = 6 or 10)
// YOLOv8 does not (classOffset = 4 directly)
final bool hasObjectness = numChannels == 6 || numChannels == 10;
final int classOffset = hasObjectness ? 5 : 4;

double _ensureProbability(double value) {
  // TFLite exports may output raw logits instead of 0-1 probabilities
  return (value < 0.0 || value > 1.0) ? _sigmoid(value) : value;
}

double _sigmoid(double x) => 1.0 / (1.0 + math.exp(-x));
```

Step 5.7 — Anchor Parsing Loop (8400 iterations)

dart

```

const double confidenceThreshold = 0.20;

for (var anchor = 0; anchor < numAnchors; anchor++) {
  final double objectness = hasObjectness
    ? _ensureProbability(_readOutput(output, 4, anchor))
    : 1.0;

  double maxClassScore = -1.0;
  for (var cls = 0; cls < yoloClasses; cls++) {
    final double classScore = _ensureProbability(
      _readOutput(output, classOffset + cls, anchor),
    );
    final double combinedScore = objectness * classScore;
    if (combinedScore > maxClassScore) maxClassScore = combinedScore;
  }

  if (maxClassScore > confidenceThreshold) {
    // Decode center-format box and un-Letterbox back to original image coords
    double cx = _toInputPixels(_readOutput(output, 0, anchor), targetSize);
    double cy = _toInputPixels(_readOutput(output, 1, anchor), targetSize);
    double w = _toInputPixels(_readOutput(output, 2, anchor), targetSize);
    double h = _toInputPixels(_readOutput(output, 3, anchor), targetSize);

    // Remove Letterbox padding to get original image coordinates
    double origCx = (cx - dx) / ratio;
    double origCy = (cy - dy) / ratio;
    double origW = w / ratio;
    double origH = h / ratio;

    double left = (origCx - origW / 2).clamp(0.0, imgW);
    double top = (origCy - origH / 2).clamp(0.0, imgH);
    double right = (left + origW).clamp(0.0, imgW);
    double bottom = (top + origH).clamp(0.0, imgH);

    rawDetections.add(Detection(
      label: 'Mosquito',
      confidence: maxClassScore,
      boundingBox: Rect.fromLTRB(left, top, right, bottom),
    ));
  }
}

```

Step 5.8 — Non-Maximum Suppression (NMS)

dart

```
rawDetections.sort((a, b) => b.confidence.compareTo(a.confidence));

for (final det in rawDetections) {
  bool keep = true;
  for (final nmsDet in nmsDetections) {
    final intersect = det.boundingBox.intersect(nmsDet.boundingBox);
    if (intersect.width > 0 && intersect.height > 0) {
      final interArea = intersect.width * intersect.height;
      final unionArea = (det.boundingBox.width * det.boundingBox.height)
        + (nmsDet.boundingBox.width * nmsDet.boundingBox.height)
        - interArea;
      final iou = interArea / unionArea;
      if (iou > 0.45) { keep = false; break; } // IoU threshold: 0.45
    }
  }
  if (keep) nmsDetections.add(det);
}
```

NMS purpose: Multiple anchors often fire on the same object. NMS keeps the highest-confidence box and suppresses overlapping boxes with IoU > 0.45.

Step 5.9 — Cap Detections

dart

```
static const int _maxDetections = 3;
final cappedDetections = nmsDetections.take(_maxDetections).toList();
```

At most 3 mosquitoes will be classified. Already sorted highest-confidence first.

Step 5.10 — Stage 2: MobileNetV3 Crop & Classify (per detection)

dart

```
for (var i = 0; i < cappedDetections.length; i++) {
  if (finalSpeciesDetections.length >= _maxDetections) break; // hard stop

  // A. Crop bounding box from original HIGH-RESOLUTION image
  final crop = img.copyCrop(original, x: x, y: y, width: w, height: h);

  // B. Resize to 224x224 (MobileNetV3 input size)
  final resizedCrop = img.copyResize(
    crop, width: 224, height: 224, interpolation: img.Interpolation.linear,
  );

  // C. Build classifier input tensor - RAW pixels [0,255], NO normalization
  // The model's built-in Rescaling(1/127.5, offset=-1) layer handles it
  final classifierInput = imageToClassifierInputTensor(resizedCrop);
  // shape: [1][224][224][3] values: [0.0, 255.0]

  // D. Run MobileNetV3-Small
  final List<List<double>> classifierOutput = List.generate(
    1, (_) => List.filled(_classifierLabels.length, 0.0),
  );
  _classifierInterpreter.run(classifierInput, classifierOutput);
  // output: [1][5] softmax probabilities

  // E. Argmax → species label
  int bestClsIdx = 0;
  double maxProb = -1.0;
  for (int c = 0; c < probs.length; c++) {
    if (probs[c] > maxProb) { maxProb = probs[c]; bestClsIdx = c; }
  }
  final String speciesLabel = _classifierLabels[bestClsIdx];

  // F. False-positive filter: discard if classified as Non-Mosquito
  if (speciesLabel.toLowerCase() == 'non-mosquito') continue;

  finalSpeciesDetections.add(Detection(
    label: speciesLabel, // Aegypti | Albopictus | Anopheles | Culex
    confidence: maxProb, // classifier probability (0.0-1.0)
    boundingBox: det.boundingBox,
  ));
}
```

```

return PredictionResult(
  detections: finalSpeciesDetections,
  imageSize: Size(original.width.toDouble(), original.height.toDouble()),
);

```

imageToClassifierInputTensor (lib/utils/image_utils.dart):

```

dart

List<List<List<List<double>>>> imageToClassifierInputTensor(img.Image image) {
  return List.generate(1, (_) =>
    List.generate(image.height, (y) =>
      List.generate(image.width, (x) {
        final pixel = image.getPixel(x, y);
        // DO NOT normalize - pass raw [0,255] floats
        // Model's Rescaling Layer: scale=1/127.5, offset=-1 → maps to [-1,1]
        return [pixel.r.toDouble(), pixel.g.toDouble(), pixel.b.toDouble()];
      }),
    ),
  );
}

```

Phase 6 — Results Screen (result_screen.dart)

Step 6.1 — initState() Side Effects

```

dart

@override
void initState() {
  super.initState();
  _prepareDisplayImage(); // async: decode + EXIF-bake for clean render
  WidgetsBinding.instance.addPostFrameCallback((_) {
    if (_hasAnyDengueVector) {
      _showBittenBottomSheet(); // auto-prompt if vector detected
    } else if (_overlayDetections.isEmpty) {
      _showNoMosquitoPrompt(); // alert if nothing detected
    }
  });
}

```

_prepareDisplayImage():

```

dart

Future<void> _prepareDisplayImage() async {
  final bytes = await widget.imagePath!.readAsBytes();
  var image = img.decodeImage(bytes);
  if (image == null) return;
  final oriented = img.bakeOrientation(image);
  setState(() {
    _displayBytes = Uint8List.fromList(img.encodeJpg(oriented, quality: 92));
  });
  // Switches from Image.file(path) to Image.memory(bytes) for stable rendering
}

```


Step 6.2 — Bounding Box Overlay Rendering

`DetectionOverlayPainter` (a `CustomPainter`) is layered over the image:

dart

```
Rect _scaleBox(Rect box, Size widgetSize) {  
  // Map original image pixel coords + widget display coords  
  final scale = widgetSize.width / originalImageSize.width;  
  return Rect.fromLTRB(  
    box.left * scale, box.top * scale,  
    box.right * scale, box.bottom * scale,  
  );  
}  
  
@override  
void paint(Canvas canvas, Size size) {  
  for (int i = 0; i < detections.length; i++) {  
    final detection = detections[i];  
    final boxColor = detection.isDengueVector ? Colors.red : Colors.green;  
    final scaledBox = _scaleBox(detection.boundingBox, size);  
    if (scaledBox.width < 2 || scaledBox.height < 2) continue; // skip degenerate boxes  
  
    // Draw bounding rect  
    canvas.drawRect(scaledBox, Paint()..color = boxColor..style = PaintingStyle.stroke..strokeWidth = 4.0);  
  
    // Draw label background + text  
    final textPainter = TextPainter(  
      text: TextSpan(text: detection.displayName, style: ...),  
      textDirection: TextDirection.ltr,  
    )..layout();  
    canvas.drawRect(labelBgRect, Paint()..color = boxColor.withOpacity(0.85));  
    textPainter.paint(canvas, Offset(scaledBox.left + 4, labelTop + 2));  
  }  
}
```

Step 6.3 — `_recordResult()` — Local Save Flow

dart

```
void _recordResult() async {
  if (_isRecorded || _isRecording) return; // guard
  setState(() => _isRecording = true);    // show loading spinner on button

  // 1. Get GPS location for the record
  final pos = await LocationService().getCurrentPosition();
  final locString = pos != null ? LocationService().toLocationString(pos) : null;

  // 2. Persist to Hive via HistoryService
  await HistoryService().addRecord(ScanRecord(
    id: DateTime.now().millisecondsSinceEpoch.toString(),
    mosquitoType: widget.mosquitoType,
    result: isPositive ? 'Dengue vector' : 'Not a dengue vector',
    confidence: widget.confidence,
    date: widget.testDate,
    imageFile: widget.imagePath,
    location: locString,
    detectionCount: widget.savedDetectionCount ?? widget.detections?.length ?? 1,
    detections: widget.detections,
    imageWidth: widget.imageSize?.width,
    imageHeight: widget.imageSize?.height,
  ));

  setState(() => _isRecorded = true);
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Result saved to history'), backgroundColor: Color(0xFF2ECC71)),
  );
  setState(() => _isRecording = false);
}
```

HistoryService().addRecord() internals:

dart

```
Future<void> addRecord(ScanRecord record) async {
  // 1. Copy image to persistent app documents directory
  final directory = await getApplicationDocumentsDirectory();
  final ext = record.imageFile!.path.split('.').last;
  final newPath = '${directory.path}/scan_${record.id}.$ext';
  final savedImage = await record.imageFile!.copy(newPath);
  // Image is now in documents dir, not the ephemeral temp dir

  // 2. Prepend to in-memory list
  _records.insert(0, recordToSave);

  // 3. Persist to Hive
  await _historyBox!.put(recordToSave.id, recordToSave.toJson());

  // 4. Enforce 12-record cap
  if (_records.length > _maxRecords) {
    final recordsToDelete = _records.sublist(_maxRecords);
    for (var r in recordsToDelete) {
      await _historyBox!.delete(r.id); // remove from Hive
      if (r.imageFile != null && await r.imageFile!.exists()) {
        await r.imageFile!.delete(); // free disk space
      }
    }
    _records.removeRange(_maxRecords, _records.length);
  }
}
```

Step 6.4 — `_shareResult()` — Cloud Upload Flow

Only visible if `_hasAnyDengueVector == true`. Hidden for non-vectors.

dart

```
void _shareResult() async {
  if (_isShared || _isSharing) return; // guard: prevent double-tap
  setState(() => _isSharing = true);

  // — Step A: Connectivity check —
  final hasInternet = await ConnectivityService().hasInternetAccess();
  if (!hasInternet) {
    _showNoInternetDialog(); // AlertDialog: no upload possible
    setState(() => _isSharing = false);
    return;
  }

  // — Step B: GPS acquisition —
  final pos = await LocationService().getCurrentPosition();
  if (pos == null) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Unable to get your location. Please enable GPS.')),
    );
    setState(() => _isSharing = false);
    return;
  }

  // — Step C: Species normalization —
  final species = widget.mosquitoType.toLowerCase().contains('aegypti')
    ? 'aegypti'
    : 'albopictus';

  // — Step D: Enqueue for Firestore upload —
  await SightingUploadService().upload(
    species: species,
    lat: pos.latitude,
    lng: pos.longitude,
    confidence: widget.confidence,
    timestamp: widget.testDate,
  );
```

```
  setState(() => _isShared = true);
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Sighting shared to the dengue map!')),
  );
  setState(() => _isSharing = false);
}
```

Phase 7 — Connectivity Check (`ConnectivityService.hasInternetAccess()`)

dart

```
Future<bool> hasInternetAccess() async {
  // Layer 1: Check OS network interface (fast, no network round-trip)
  final List<ConnectivityResult> results = await Connectivity().checkConnectivity();
  if (results.contains(ConnectivityResult.none)) {
    return false; // No interface (airplane mode, WiFi off, mobile off)
  }

  // Layer 2: Verify actual internet routing (DNS + HTTP, 5s timeout)
  try {
    final response = await http.head(
      Uri.parse('https://tile.openstreetmap.org'),
    ).timeout(const Duration(seconds: 5));
    return response.statusCode < 500;
    // statusCode < 500 means the server responded (even 404 = internet is up)
  } catch (_) {
    // Timeout, DNS resolution failure, or TLS error
    return false; // Interface is up but routing is broken (captive portal, ISP issue)
  }
}
```

Why two-layer check? A device can have WiFi connected but no internet (e.g., captive portal). The HTTP HEAD to OpenStreetMap confirms end-to-end routing without downloading any data.

Phase 8 — Location Acquisition `LocationService.getCurrentPosition()`

dart

```
Future<Position?> getCurrentPosition() async {
  // 1. Check if GPS hardware/service is enabled
  bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
  if (!serviceEnabled) return null; // GPS turned off in device settings

  // 2. Check app permission
  LocationPermission permission = await Geolocator.checkPermission();

  // 3. Request permission if not yet granted
  if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
    if (permission == LocationPermission.denied) return null; // User denied
  }

  // 4. Handle permanently denied (user checked "Never ask again")
  if (permission == LocationPermission.deniedForever) return null;

  // 5. Acquire position with platform-specific settings
  try {
    LocationSettings locationSettings;
    if (defaultTargetPlatform == TargetPlatform.android) {
      locationSettings = AndroidSettings(
        accuracy: LocationAccuracy.high,
        timelimit: Duration(seconds: 10),
        forceLocationManager: true, // bypass Google Play Services for compatibility
      );
    } else {
      locationSettings = LocationSettings(
        accuracy: LocationAccuracy.high,
        timelimit: Duration(seconds: 10),
      );
    }
    return await Geolocator.getCurrentPosition(locationSettings: locationSettings);
  } catch (e) {
    return null; // Timeout or hardware error
  }
}
```

```
// Helpers:
String toLocationString(Position p) => '${p.latitude},${p.longitude}';
LatLng? parseLocationString(String? s) {
  if (s == null || !s.contains(',')) return null;
  final parts = s.split(',');
  return LatLng(double.parse(parts[0]), double.parse(parts[1]));
}
```

Phase 9 — Upload Queue & Firestore Write (UploadQueueService)

dart

```
// SightingUploadService.upload() + thin wrapper, swallows exceptions
Future<void> upload({...}) async {
  try {
    await UploadQueueService().enqueue(...);
  } catch (e) {
    debugPrint('Failed to enqueue sighting: $e');
    // intentional silent fail - local flow continues
  }
}
```

UploadQueueService.enqueue():

dart

```
Future<void> enqueue({String species, double lat, double lng,
                    double confidence, DateTime timestamp}) async {
  // 1. Generate deduplication key
  final key = _generateKey(species: species, lat: lat, lng: lng, timestamp: timestamp);
  // key format: "aegypti-3.140-101.686-2026-07-20T10:07:00.000"

  // 2. Deduplication check
  if (_queueBox.containsKey(key)) {
    debugPrint('Sighting already in queue (deduplicated): $key');
    return;
  }

  // 3. Write to Hive
  final item = {
    'species': species, 'lat': lat, 'lng': lng,
    'confidence': confidence,
    'timestamp': timestamp.toIso8601String(),
    'retryCount': 0,
    'nextRetryTime': DateTime.now().toIso8601String(),
  };
  await _queueBox.put(key, item);

  // 4. Immediate drain attempt
  _drainQueue();
}
```

UploadQueueService._drainQueue() :

dart

```
Future<void> _drainQueue() async {
  if (_isDraining || _queueBox.isEmpty) return;
  _isDraining = true;

  try {
    // 1. Collect items due for retry (up to 10)
    final now = DateTime.now();
    final itemsToProcess = <String, Map<dynamic, dynamic>>{};
    for (var key in _queueBox.keys) {
      final item = _queueBox.get(key) as Map<dynamic, dynamic>;
      final nextRetryTime = DateTime.parse(item['nextRetryTime'] as String);
      if (now.isAfter(nextRetryTime) || now.isAtSameMomentAs(nextRetryTime)) {
        itemsToProcess[key as String] = item;
        if (itemsToProcess.length >= 10) break;
      }
    }
  }
  if (itemsToProcess.isEmpty) return;

  // 2. Connectivity check (avoids wasting Firestore quota on known-offline)
  final hasInternet = await ConnectivityService().hasInternetAccess();
  if (!hasInternet) return;

  // 3. Ensure Firebase Auth session
  User? user = FirebaseAuth.instance.currentUser;
  if (user == null) {
    final authResult = await FirebaseAuth.instance.signInAnonymously();
    user = authResult.user;
  }
}
```



```

// 4. Build and commit Firestore WriteBatch (atomic multi-doc write)
final db = FirebaseFirestore.instance;
final batch = db.batch();
for (var entry in itemsToProcess.entries) {
  final item = entry.value;
  final docRef = db.collection('sightings').doc(); // auto-generated ID
  batch.set(docRef, {
    'species': item['species'],
    'lat': item['lat'],
    'lng': item['lng'],
    'confidence': item['confidence'],
    'timestamp': Timestamp.fromDate(DateTime.parse(item['timestamp'] as String)),
    'userId': user?.uid ?? 'unknown',
  });
}

try {
  await batch.commit();
  // 5a. Success: remove from queue
  for (var key in itemsToProcess.keys) {
    await _queueBox.delete(key);
  }

} on FirebaseException catch (e) {
  await _handleDrainError(itemsToProcess, e);
} catch (e) {
  await _handleDrainError(itemsToProcess, Exception(e.toString()));
}

} finally {
  _isDraining = false;
}
}

```

`_handleDrainError()` — Retry Strategy:

dart

```
Future<void> _handleDrainError(items, error) async {
  bool isPermanent = false;
  if (error is FirebaseException) {
    // Permanent errors – retrying won't help
    if (['permission-denied', 'invalid-argument', 'unauthenticated'].contains(error.code)) {
      isPermanent = true;
    }
    // Transient: 'resource-exhausted' (quota), 'unavailable' (503) → retry
  }

  for (var entry in items.entries) {
    if (isPermanent) {
      // Dead letter: move item out of main queue, preserve for debugging
      await _deadLetterBox.put(entry.key, entry.value);
      await _queueBox.delete(entry.key);
    } else {
      // Exponential backoff: 2s → 4s → 8s → 16s → ... → 60s (capped)
      int retryCount = (item['retryCount'] as int? ?? 0) + 1;
      int backoffSeconds = math.min(1 << retryCount, 60);
      item['retryCount'] = retryCount;
      item['nextRetryTime'] = DateTime.now()
        .add(Duration(seconds: backoffSeconds)).toIso8601String();
      await _queueBox.put(entry.key, item);
    }
  }
}
```